

RCS-VRF Protocol — Comprehensive Design Notes

A consolidated reference covering everything substantive from the recent design conversations: the certification math chain, the adversary model, the Leg 1 redesign, the workflow architecture, and all open items.

Table of Contents

1. Project context and pipeline
 2. The certification math chain (Legs 3-6)
 3. The adversary model
 4. $Q_{\min}$: what it is, how it's calculated, where we are
 5. What we're actually certifying
 6. The Leg 1 redesign (replacing EC-VRF)
 7. Workflow architecture: schedule-driven vs request-then-fulfill
 8. VRF architectural lineage and comparisons
 9. Mehdi's deck — what he added beyond our earlier conversation
 10. Key corrections made during the session
 11. Documents produced
 12. Outstanding items and the critical path
 13. Reading resources
-

1. Project context and pipeline

The project: A subnet that combines quantum random circuit sampling (RCS) with classical verification to produce **certified randomness** — a product that classical RNGs cannot deliver because they cannot certify quantum origin or prove security against an adversarial provider.

Pipeline (seven legs):

- **Leg 1** — Seed generation. Produces an unpredictable, unbiasable, verifiable seed for each round.
- **Leg 2** — Parameter precommitment. Locks circuit-generation parameters before the seed is used.
- **Leg 3** — Circuit generation. Deterministically expands the seed into Haar-like random circuits via SHAKE-256.
- **Leg 4** — Quantum sampling. The (untrusted) quantum server runs the circuits and returns samples.
- **Leg 5** — Verification. Classical validators check F_{XEB} , timing, and certify entropy.
- **Leg 6** — Extraction. Toeplitz/Trevisan extractor turns accepted samples into uniform-looking certified bits.
- **Leg 7** — Anchoring. Public/private randomness anchored in a tamper-evident audit log, served to consumers.

The four claims the protocol delivers:

1. *Statistical near-uniformity* of output bits (indistinguishable from uniform to a polynomial-time adversary).
2. *Quantum origin* (the randomness comes from physical measurements, not classical computation).
3. *Adversary-robust* (claims 1 and 2 hold even if the quantum server is dishonest).
4. *Publicly verifiable* (anyone with the audit log can re-derive all numerical claims).

On claim 3 specifically: **adversary-robust** means the protocol’s claims about uniformity and quantum origin survive even when the server is actively trying to cheat — because the math chain forces a floor on honest quantum execution that no cheating strategy within the threat model can defeat. This is the property that makes certified randomness non-trivial: without it, claims 1 and 2 would just be promises that depend on trusting the server. With it, they become cryptographic guarantees that survive a hostile counterparty.

2. The certification math chain (Legs 3-6)

This section traces the math from raw quantum measurements to a final bag of certified random bits.

2.1 Haar randomness

Haar measure is the unique uniform distribution over unitary matrices. A Haar-random unitary U on n qubits is sampled uniformly from $U(2^n)$.

Why it matters: a Haar-random unitary has no structure for a classical attacker to exploit. Output statistics become universal and tractable.

The catch: truly Haar-random unitaries require exponentially many gates to build. They’re infeasible. We need an approximation.

2.2 t -designs and RCS

A **unitary t -design** is a finite ensemble of unitaries reproducing the first t moments of the Haar distribution. For F_XEB and collision entropy, we need only the **second moment** — so a 2-design suffices.

Random Circuit Sampling (RCS) is the engineering recipe: alternating layers of random single-qubit gates and entangling two-qubit gates. After roughly n layers, the resulting unitary is statistically indistinguishable from Haar to second-moment tests. This is what the QPU actually runs.

2.3 Porter-Thomas distribution

For Haar-random (or 2-design) circuits, output probabilities $p(x) = |\langle x|U|0^n \rangle|^2$ follow:

$$\Pr[N \cdot p(x) \in [s, s + ds]] = e^{-s} ds, \quad N = 2^n.$$

The single number that drives everything:

$$\sum_x \mathbb{E}[p(x)^2] = \frac{2}{N}.$$

This is the **second moment** of Porter-Thomas. Three reasons it’s “magic”:

- **Universal** across all 2-designs.
- **Statistically checkable** from samples (which is what F_XEB does).
- **Feeds the per-round entropy formula** directly.

2.4 F_XEB — the linear cross-entropy benchmark

For an audit subset of m circuits, the verifier classically simulates each and computes:

$$F_{\text{XEB}} = \frac{N}{m} \sum_{i=1}^m p_{C_i}(x_i) - 1.$$

Four reference cases:

- Uniform noise / blind spoof $\rightarrow F_{\text{XEB}} \approx 0$
- Real noisy quantum machine $\rightarrow F_{\text{XEB}} \approx \phi$ (fidelity, ~ 0.4 on current hardware)
- Perfect quantum $\rightarrow F_{\text{XEB}} = 1$
- Cherry-picking spoof $\rightarrow F_{\text{XEB}} \gg 1$

Liu et al. threshold: $\chi = 0.3$.

2.5 Acceptance check — one-sided (Liu) vs two-sided (our addition)

Liu uses a one-sided check: accept iff $F_{\text{XEB}} \geq \chi$.

Their argument: a compute-rational adversary won't score $\gg 1$ (that requires *more* honest quantum work, not less). So no upper bound is needed.

Our v0.1+ uses a two-sided check: $\chi_{\text{low}} \leq F_{\text{XEB}} \leq \chi_{\text{high}}$.

This is a **defence-in-depth choice**, not a Liu prescription. It catches a different threat class: wasteful adversaries, heavy-output generation, adversaries who circumvent the compute-budget assumption. The lower bound is the load-bearing one; the upper bound is our conservative addition.

2.6 Timing — the second half of the proof

F_{XEB} alone proves correctness but not quantum origin. A patient classical attacker could simulate every circuit and pass.

The per-circuit response deadline ($t_{\text{threshold}} \sim 2$ seconds for Liu's setup) closes the classical-faking route. Quantum hardware clears it naturally; classical simulation of $n = 56$ circuits takes hours per circuit.

Status in our code: timing is *stubbed* — recorded in the audit log but not enforced. The math chain assumes enforcement; engineering is owed. This is the most important engineering gap.

2.7 Per-round collision entropy: $n - 1$

The collision (2-Rényi) entropy of the Porter-Thomas distribution:

$$H_2 = -\log_2 \sum_x p(x)^2 = -\log_2(2/N) = n - 1.$$

For $n = 56$: 55 bits per honest quantum round.

This is the cleanest piece of the chain. Derived from first principles. No convention freedom. Same across Liu, Aaronson-Hung, and every RCS-based certified-randomness protocol.

2.8 $Q_{\min}$ — minimum forced quantum rounds

(Detailed in Section 4.)

$$Q_{\min} = \min\{Q : \varepsilon_{\text{adv}}(Q, \chi) \geq 4\varepsilon_s\}.$$

For Liu’s published parameters: $Q_{\min} = 1297$ out of $M = 30,010$.

2.9 Certified min-entropy (Liu Theorem 1)

$$H_{\min}^{\text{cert}} = Q_{\min} \cdot (n - 1) - \log_2(1/\varepsilon_s).$$

For Liu’s parameters: $1297 \times 55 - 21.9 \approx \mathbf{71,313}$ bits.

Our code reproduces this exactly given Liu’s $Q_{\min}$.

2.10 Toeplitz extraction with the Leftover Hash Lemma (Liu Corollary 7)

Take the raw samples (a bitstring of $M \cdot n$ bits) and a fresh independent random seed (drand pulse in our implementation). Multiply by a Toeplitz matrix T over $\text{GF}(2)$:

$$\mathbf{output} = T \cdot \mathbf{raw_bits} \pmod{2}.$$

The output length is bounded by:

$$\ell \leq Q_{\min} \cdot (n - 1) - 3 \log_2(1/\varepsilon_{\text{sou}}) - 2.$$

For Liu’s parameters: $\ell \approx \mathbf{71,273}$ bits. The 40-bit gap between 71,313 and 71,273 is the cost of quantum-secure extraction (against quantum side information).

2.11 What’s “the same as Liu” vs “still owed”

Exactly the same as Liu:

- F_XEB formula and computation
- Acceptance check lower bound ($\chi = 0.3$)
- Certified min-entropy formula (Theorem 1)
- $H_2 = n - 1$ per round (derived from first principles)
- Toeplitz + LHL extraction (Corollary 7)

Our deliberate additions/deviations:

- Two-sided F_XEB check (defence-in-depth)

Still owed:

- $Q_{\min}$ calibration (structure right, three numerical conventions need reconciliation against Liu’s Zenodo code)
- Timing enforcement (recorded but not enforced)

Notable simplification:

- Liu’s edge-coloured random graph for circuit connectivity $\rightarrow$ our random perfect matching per layer. Fine at toy scale; would need to be upgraded to match real QPU hardware topology at production scale.

3. The adversary model

3.1 Who is “the adversary” in our protocol

Three layers, mapped to different protocol defences:

Primary adversary — the quantum-hardware vendor (miner). The vendor (IonQ, Quantinuum, etc.) might fake quantum work to save costs. Defended by F_{XEB} , timing, $Q_{\min}$ floor.

Secondary adversary — the operator (us). The protocol operator might bias the seed or the audit log to favour themselves. Defended by the Leg 1 commit-reveal + beacons design and the publicly-verifiable audit log.

Tertiary adversary — external attackers. Network attackers, gaming customers, regulators, coordinated multi-party attackers. Defended by signed beacons, sorted pool aggregation, multi-source independence.

The protocol’s defining property is that **it doesn’t rely on trust in any single party**. The math holds even when each role is played adversarially.

3.2 Liu’s specific adversary model

Liu analyses the protocol against a “*restricted realistic near-term adversary*” with:

1. **Perfect-fidelity quantum hardware** (strongest version of the assumption — eliminates “the cheater’s quantum machine is noisy” as a defence).
2. 1.1×10^{18} **FLOPS classical compute** (current top supercomputers — Frontier, El Capitan, Aurora).
3. **Best-known classical attacks on F_{XEB}** (encoded in the constant c_{eff}).
4. **Same per-circuit deadline as honest server** (the binding constraint).

This is **one adversary with both quantum and classical resources** — not two adversaries. It’s the cryptographic worst-case convention: prove security against the strongest version of the threat.

3.3 The adversary’s strategy

Given a perfect quantum machine, a bounded classical budget, and a deadline, the worst-case cheater **mixes** honest quantum rounds (real measurements) with classical spoofs (high-probability strings computed within the deadline) to *minimise* quantum work while still passing F_{XEB} .

The classical faking budget within the deadline:

$$\Phi_A \approx \frac{A \cdot t_{\text{threshold}}}{B}$$

where B is the cost of faking one circuit’s F_{XEB} contribution (encoded by c_{eff}).

3.4 Why one adversary with both capabilities isn’t a contradiction

A cheater with a perfect quantum machine *could* just use it for all M circuits — they’d score $F_{\text{XEB}} = 1$ and pass. But then they’d be running the honest protocol and producing real certified randomness. They wouldn’t be “cheating.”

The relevant question is: *given an adversary trying to skip quantum work, how much can they skip?* The answer is bounded by the classical budget within the deadline. The quantum machine is a “free resource” the adversary could use; the **binding constraint is the classical budget + deadline**, which together force a minimum of $Q_{\min}$ honest quantum rounds.

3.5 What Liu’s model does NOT cover

- A future fault-tolerant quantum adversary that can natively sample at deadline pace.
- A future classical adversary with $\geq 10^{20}$ FLOPS.
- An adversary who discovers a fundamentally better classical algorithm for spoofing F_{XEB} (would invalidate c_{eff}).
- Side-channel attacks (network timing, power analysis, etc.).
- Device-tampering attacks on the verifier.
- Coordinated multi-party attacks outside the modelled structure.

The bound is a **near-term** security claim, not an asymptotic one. The model is *time-limited* and *current-best-estimate*. Each customer use case should ask: “is the near-term security horizon enough for me?”

3.6 Pros and cons of Liu’s adversary choice

Pros:

- Non-trivially large bound (71,313 certified bits/batch is a real product).
- Matches who actually exists today.
- Uses best-known attacks honestly.
- Auditable and publicly criticisable.
- Composes well with downstream applications (similar to TLS/AES “current-best” framing).
- Gives the adversary perfect quantum hardware (robust against “your hardware is broken” assumption).
- Tunable per customer (calibrated `compute_q_min` enables parameter-regime generalisation).

Cons:

- Time-limited; bound weakens as classical compute grows.
- Depends on c_{eff} remaining best-known.
- Doesn’t cover side-channels, tampering, or future quantum attackers.
- Hard to communicate to non-cryptographers (the precise claim is nuanced).

4. $Q_{\min}$: what it is, how it’s calculated, where we are

4.1 What $Q_{\min}$ is, precisely

$Q_{\min}$ is the **minimum number of honest quantum rounds** the worst-case adversary cannot avoid, given:

- Their quantum and classical resources (Liu’s adversary model).
- The acceptance threshold χ .
- The protocol’s target failure probability ε_s .

It’s a **floor**: even with everything at their disposal, the adversary couldn’t have passed $F_{\text{XEB}} \geq \chi$ using fewer than $Q_{\min}$ honest quantum rounds.

The same number is the adversary’s lower bound *and* our certification’s floor. They meet at $Q_{\min} = 1297$ for Liu’s parameters.

4.2 Liu’s exact definition (Lemma 2)

$$Q_{\min} = \min\{Q : \varepsilon_{\text{adv}}(Q, \chi) \geq 4\varepsilon_s\}.$$

where $\varepsilon_{\text{adv}}(Q, \chi)$ is the upper bound on the probability that an adversary executing exactly Q honest quantum rounds (and faking $M - Q$) can pass the F_XEB test at threshold χ .

4.3 How it’s computed

The calculation has six steps:

1. Bound the adversary’s classical faking budget $\Phi_A = A \cdot t_{\text{threshold}}/B$.
2. Model the adversary’s mixed strategy (honest fraction δ vs faked fraction).
3. Apply hypergeometric-tail bound for the audit subset’s F_XEB statistics.
4. Optimise over δ — worst-case adversary.
5. Account for the ε_2 smoothing tail.
6. Sweep Q downward to find the smallest one satisfying the security condition.

4.4 Current state — calibrated and end-to-end verified

(Section header revised: was “uncalibrated, not a placeholder” — updated after the calibration was completed against Liu’s reference notebook.)

What we have:

- Full implementation of Liu’s reference algorithm in `leg5_qmin_liu.py`.
- Exact reproduction of the Liu notebook output: $Q_{\min} = 1297$, $H_{\min}^{\text{cert}} = 71,313.07$ bits, extractable $\ell = 71,273.21$ bits — match to six decimal places.
- End-to-end verified against Liu’s published experimental data (`aggregated_probs.npy`, 1,522 precomputed validation probabilities). F_XEB = 0.32 reproduces Liu’s published value; downstream chain reproduces the paper’s headline numbers.

The three numerical conventions that were reconciled:

1. **δ selection.** Our previous implementation used a 400-point numerical grid; Liu uses the closed-form Chernoff-optimal $\delta = \sqrt{3 \log(2/\varepsilon_{\text{sou}})/\Phi_A}$. Closed-form is structurally cleaner and removes grid-resolution sensitivity.
2. **Soundness budget split.** Our previous implementation used a combined threshold of $4 \cdot \varepsilon_s = \varepsilon_{\text{sou}}$ on the sum of Chernoff and hypergeometric tails; Liu splits the budget as $\varepsilon_{\text{sou}}/2$ on each tail separately. The half-budget split is what’s actually in the reference code.
3. **The effective compute factor.** Liu’s notebook uses $A = 4 \times \text{FRONTIER}$ theoretical peak ($= 8 \times 10^{18}$ FLOPS) with $\text{eff} = 0.5$, giving an effective adversary compute of 4 exaFLOPS. This is roughly the aggregate of the world’s top four supercomputers at sustained efficiency. Earlier attempts had the right numerical product but the wrong factorisation; the principled factorisation makes the modelling assumptions explicit.

All three are documented in the calibrated module `leg5_qmin_liu.py` and verified by the regression test `test_qmin_calibrated.py` and the end-to-end script `verify_against_liu_data.py`.

4.5 Why we deliberately didn't fudge it

It would have been trivial to tune one or two constants to hit 1297 exactly without doing the principled reconciliation work. We refused at every stage. Fitting parameters to the answer would have made the regression test a tautology.

What we did instead: marked the assertion `xfail`, recorded honestly that we *hadn't* derived 1297 yet, and waited until Liu's reference notebook was available before doing the real reconciliation. When the notebook came, the test flipped green for the right reason: we plugged in Liu's actual conventions ($\text{eff} = 0.5$, $A = 4 \times \text{FRONTIER_theoretical}$, closed-form δ , half-budget threshold split) and got Liu's actual number from first principles.

The discipline matters because it's the difference between "we built the protocol" and "we tuned constants to match a target." The first claim is auditable; the second isn't.

4.6 What calibration unlocked

- **Math chain closed end-to-end** — every test passes for the right reason.
- **Parameter-regime generalisation** — plug in any n , M , χ , ε_s , A , $t_{\text{threshold}}$ and get a derived $Q_{\min}$.
- **Customer-specific tightening or loosening** — stricter adversary for regulated use cases, lighter for casual ones. See Section 4.8 for the empirical effect of changing adversary power.
- **Credibility asset** — independently-verifiable implementation, not transcription of paper constants. Run `verify_against_liu_data.py` and the result is reproducible by any reviewer with the Zenodo archive.

4.7 What it does NOT unlock

Calibration doesn't change the *security* of the protocol in practice. The blockers for real product are:

- **Real QPU access (Leg 4)** — without it, "quantum-in-origin" is unmet.
- **Timing enforcement (Leg 5)** — without it, the deadline portion of the proof is unrealised.

Both are hardware/engineering items, not math items. Calibration is the last math item; the real protocol items are downstream.

4.8 How $Q_{\min}$ depends on adversary power

A useful precision pass: the symbol $Q_{\min}$ is doing **two jobs at once**, and these can sound contradictory if not held carefully.

Meaning A — The certified floor. In the certification claim, $Q_{\min}$ is the *lower bound* on the number of rounds that must have been honestly quantum, given the audit passed. Higher $Q_{\min}$ means a stronger claim ("lots of the rounds had to be real quantum"), which translates into more certified entropy and more extractable bits.

Meaning B — The boundary value. In the security computation, $Q_{\min}$ is found by binary search over a function of Q . It converges to the boundary value where the adversary's pass probability equals $\varepsilon_{\text{sou}}/2$. Below this Q , the adversary can pass; at or above it, the adversary fails with high confidence.

The two meanings are the same number. Liu's binary search finds the boundary; when the audit *does* pass, we conclude the adversary must have been on the secure side of that boundary, and the boundary value becomes the certified floor. Same number, two perspectives.

Direction of the dependence on adversary power The relationship is **inverse**:

- **A weaker adversary** has less classical compute $\rightarrow$ can fake fewer rounds $\rightarrow$ needs more honest quantum rounds to push the audit over $\chi \rightarrow$ the boundary Q where they transition from “can’t pass” to “can pass” is **larger** $\rightarrow$ we can certify “at least [large number] rounds were quantum” when the audit passes.
- **A stronger adversary** has more classical compute $\rightarrow$ can fake more rounds $\rightarrow$ can pass the audit with fewer honest quantum rounds because their faking compensates $\rightarrow$ the boundary Q is **smaller** $\rightarrow$ we can only certify “at least [small number] rounds were quantum” — because the strong adversary could plausibly have faked the rest.

More adversary power $\rightarrow$ smaller $Q_{\min} \rightarrow$ less certified randomness.

Empirical illustration from Liu’s notebook Liu’s `Table2-bounds-on-extractable-entropy.ipynb` directly sweeps over adversary strength (in multiples of Frontier theoretical peak) and reports the certified min-entropy rate $H/(M \cdot n)$:

Adversary power	$\varepsilon_{\text{sou}} = 10^{-6}$
1 $\times$ Frontier	rate 0.12
2 $\times$ Frontier	rate 0.09
4 $\times$ Frontier	rate 0.04 $\leftarrow$ Liu’s choice
6 $\times$ Frontier	rate ≈ 0 (protocol breakdown)
8 $\times$ Frontier	rate ≈ 0 (protocol breakdown)

(All entries assume $\text{eff} = 0.5$, i.e., 50% sustained efficiency on the theoretical peak.)

At 6 $\times$ Frontier and above, the certified rate collapses to zero — the assumed adversary is strong enough to potentially fake the entire batch, so the protocol cannot certify any randomness. This is the **protocol breakdown point**: too much assumed adversary power and the security claim collapses.

Operational implication for RCS-VRF Choosing our own adversary model is choosing where the line sits. The tradeoff:

- **Defending against a stronger adversary** (conservative) $\rightarrow$ smaller $Q_{\min} \rightarrow$ fewer certified bits per run $\rightarrow$ safer to claim, less product to sell.
- **Defending against a weaker adversary** (less conservative) $\rightarrow$ larger $Q_{\min} \rightarrow$ more certified bits per run $\rightarrow$ more product to sell, but the claim is only valid if the adversary really is bounded that way.

This is not a *purely technical* decision. It depends on:

- **Customer expectations** — what level of assurance does the customer demand?
- **Deployment timeframe** — longer deployment means we must defend against *future* attackers, not just present-day ones. Classical algorithms improve. Compute grows. An adversary model good for 2026 may be wrong by 2030.
- **Audit story rigour** — a customer planning to point regulators at our certification needs a defensible adversary model; an internal engineering customer may tolerate less conservatism.

Liu’s choice of “4 $\times$ Frontier $\times$ 50%” $\rightarrow$ 4 exaFLOPS effective is moderately conservative for 2025 deployment. We have at least three reasonable options for RCS-VRF:

1. **Inherit Liu’s model verbatim.** Easiest. Lets us cite the Nature paper as precedent. Same numbers we’ve validated.

2. **Tighten the model.** E.g., assume $8 \times \text{Frontier}$ or $16 \times \text{Frontier}$ (closer to the protocol breakdown point) to defend against a longer-horizon adversary. Cheaper certified bits per run; stronger claim.
3. **Loosen the model.** E.g., $2 \times \text{Frontier}$ or $1 \times \text{Frontier}$ (single sustained Frontier). Cheaper to defend against; more certified bits per run; weaker claim.

There’s no universally correct answer. The decision belongs in the conversation with Colton (who has the customer-side context) and Mehdi (who has the cryptographic-conservatism context), not in this document. What this section does is make the decision space explicit so the conversation can be precise about what’s being chosen.

4.9 Summary of $Q_{\min}$ status (as of end-of-session)

- Implementation: faithful to Liu’s reference (`leg5_qmin_liu.py`).
- Calibration: matches Liu’s notebook output exactly given Liu’s parameters.
- End-to-end verified: $F_{\text{XEB}} = 0.32$, $Q_{\min} = 1297$, $H = 71,313$, $\ell = 71,273$ reproduced from Liu’s `aggregated_probs.npy` data.
- Open: choosing our own adversary model (parameter decision, not math work).

5. What we’re actually certifying

The product, stated precisely:

We certify that these bits are statistically indistinguishable from uniform random bits, that their unpredictability comes from physical quantum measurements rather than classical computation, that this was the case even if the quantum hardware provider was trying to cheat, and that any party with the audit log can verify all of this without trusting us.

5.1 The four specific claims

1. **Indistinguishable from uniform** to any computationally-bounded adversary in our threat model. Statistical distance from uniform $\leq \varepsilon_{\text{sou}} = 10^{-6}$.
2. **Quantum origin:** unpredictability physically generated by quantum measurement, not by classical computation. No deterministic explanation under the laws of quantum mechanics.
3. **Adversary-robust:** claims 1 and 2 hold even if the server cheats maximally, because at least $Q_{\min}$ rounds must have been honest.
4. **Publicly verifiable:** the audit log lets any third party re-derive everything with only classical compute.

5.2 What we are NOT certifying

- *Not “perfect” randomness* — we’re ε -close to uniform, not exactly.
- *Not secure against all adversaries* — only the modelled threat class.
- *Not server honesty* — we explicitly assume the server might cheat.
- *Not per-bit guarantees* — bulk distributional claim, not “this bit is quantum.”
- *Not delivery timing* — a missed deadline means abort, not insecure output.
- *Not suitability for any particular application* — customer is responsible for fitness-for-use.

5.3 Why this differs from any classical RNG

Claims 1 and 4 a classical RNG service can technically deliver. **Claims 2 and 3 are unique to quantum-certified-randomness as a product category.** They justify the whole expensive apparatus (QPU, verification chain, audit log). Without claims 2 and 3, a classical RNG would be cheaper and equally good.

6. The Leg 1 redesign (replacing EC-VRF)

6.1 The problem

Leg 1 (seed generation) initially used **EC-VRF (Ed25519)** — the textbook tool. On closer reading, it carried three independent problems for our setting:

Problem 1: Not quantum-secure. EC-VRF rests on the elliptic-curve discrete log problem, which Shor’s algorithm breaks in polynomial time on a sufficiently capable quantum computer. We’re a quantum-enabled protocol; standing the seed layer on assumptions our own technology could one day break is internally inconsistent.

Problem 2: Structurally single-keyholder. Per VRF evaluation there is exactly one secret key, held by one party. This forces a choice between:

- *Centralised trust* — one operator holds the key, becomes a single point of trust.
- *Full validator-network* — many keyholders running parallel VRFs, requiring registration, Sybil resistance, incentives, and slashing (essentially Ethereum-PoS-scale infrastructure for a small task).

This was where we initially leaned toward multi-participant; the validator-network overhead was the deal-breaker.

Problem 3: MEV exposure in deployment. In any operator-served VRF (Chainlink-style request-then-fulfill), there’s a private knowledge window between request and fulfillment during which the operator can front-run, selectively delay, or leak the upcoming value. Structural to the workflow, not the cryptography.

6.2 Why PQ-VRF doesn’t rescue

Switching to a post-quantum VRF (LB-VRF, X-VRF) fixes only Problem 1. Problems 2 and 3 are architectural and workflow-level, not cryptographic. PQ-VRFs are also research-grade — no NIST standard, KB-scale proofs, X-VRF’s uniqueness proof was broken at FC 2024-25. They’re a hopeful direction, not a ready drop-in.

Conclusion: drop the VRF abstraction entirely.

6.3 The replacement: hash-based commit-reveal + external beacons + puzzle + chained transcript

The redesign combines four independent contribution channels under a canonical hash:

1. **Operator commit-reveal.** Protocol operator commits to a value, then reveals after the commit window closes.
2. **Public pool commit-reveal.** Anyone can submit a commitment during the window, then reveal.
3. **External beacons:** drand (federated threshold-BLS) and NIST (centralised, signed pulses) — pulses postdating the commit window’s close.

4. **PoW-style public puzzle** (per Mehdi's deck): after commits close, anyone in the world can solve a hash puzzle $H(\text{puzzle} \parallel \text{nonce}) < \text{target}_T$. First valid solution before cutoff is included.
5. **Chained transcript** (per Mehdi's deck): each round's seed depends on the previous round's finalized hash, forming a hash-chain integrity link across rounds.

Canonical seed transcript (Mehdi's slide 18):

```
seed = H( "quip-qvrf-seed-v1" || round_id || parameter_commitment
          || commitment_root || valid_reveals_root || non_reveal_root
          || drand_pulse || nist_pulse || puzzle_result
          || prev_finalized_round_hash )
```

Important note on what came from where: before Mehdi's deck, our shared design was just commit-reveal + external beacons (four channels). The deck *added* the puzzle and the chained transcript on top.

6.4 Security properties of the new design

- **Quantum-resistant:** only hash functions (Grover-bounded — 128-bit post-quantum security from SHAKE-256 with 32-byte output) and verifiable beacon signatures. No discrete-log primitives.
- **No single keyholder:** no party holds a secret key whose secrecy determines the seed's unpredictability. (Signing keys exist for *authentication* of contributions, separately from *unpredictability* of the seed.)
- **Multi-source resilience:** honest as long as *any one of the channels* is honest in a given round.
- **No MEV window:** schedule-driven workflow; no party holds a computed-but-undelivered result.
- **Publicly verifiable:** anyone can re-derive the seed from the audit log; no proof object needed.

6.5 The hash primitive question — still open

Our code uses SHAKE-256 throughout. Mehdi's deck (slide 6, v6 annotation) flags "*H/SHAKE256 question still open*" — the choice between SHA-3-256 (fixed-output) and SHAKE-256 (extendable-output) for the seed canonical rule is not yet pinned down. Both are quantum-resistant; difference is flexibility. **Open question for next design meeting.**

7. Workflow architecture: schedule-driven vs request-then-fulfill

7.1 The two workflows

Request-then-fulfill (Chainlink-style):

1. User publicly requests randomness.
2. Operator computes the result locally with their secret key.
3. *Private window:* operator alone knows the result.
4. Operator publishes result + proof.

Defining feature: one party knows the result before everyone else.

Schedule-driven (our redesign):

1. Protocol runs on a fixed timer.
2. Multiple parties commit values before any reveals.

3. Beacon pulses arrive on external schedule, postdating commits.
4. Reveals happen openly in a fixed window; all visible simultaneously.
5. Final seed computed by everyone simultaneously from public inputs.

Defining feature: no party ever holds a computed-but-undelivered result.

7.2 The MEV question

“MEV window” = the time gap between a party knowing a result and publishing it, during which they can front-run, selectively delay, leak, or reorder.

In Chainlink-style fulfillment, this window exists and is measured in seconds. In schedule-driven workflow, the window doesn’t exist because the output materialises publicly the instant the last input lands. **Schedule-driven kills fulfillment-window MEV at the root.**

7.3 Precision about MEV

“Schedule-driven dodges MEV” is correct for the **fulfillment-window class** specifically. MEV in general is broader. Different MEV classes need different defences:

MEV class	What it is	Our defence
Fulfillment-window	Operator holds result privately	Schedule-driven workflow
Last-revealer bias	Last party to reveal sees others’ values first	Commit-reveal mechanism (commitments locked before reveals)
Predictability	Inputs known in advance enable forecasting	Beacon postdating (pulses unknown at commit time)
Block-builder	Downstream consumers re-ordered by builders	Out of scope (consumer’s problem)
Validator-coordination	Multi-validator collusion	Threshold cryptography (drand) or honest-majority (Polkadot)

Our design closes the first three. The fourth is the consumer’s concern. The fifth doesn’t apply because we’re not running a validator network.

8. VRF architectural lineage and comparisons

8.1 How close is our redesign to a VRF?

	Role	Mechanism
Same as VRF	100%	Public-input → pseudo-random output, publicly verifiable
Different from VRF	—	No secret key, no proof object, multi-source, hash-based

Cleanest framing: *“We’re not building a VRF. We’re solving the same problem a VRF solves, with a different and better-suited primitive for our setting.”*

The architectural lineage is closer to **Algorand’s seed beacon** (multi-party deterministic combining) or **drand itself** (federated threshold cryptography) than to Chainlink VRF.

8.2 Comparison table

	Chainlink VRF	Polkadot BABE	drand	NIST	Our redesign
Workflow	Request-then-fulfill	Schedule-driven	Schedule-driven	Schedule-driven	Schedule-driven
Trust model	Single keyholder	Threshold of validators	Threshold of federation	Trust NIST	Any one of four sources honest
Quantum-secure	No	No (uses VRF)	Partially (BLS)	Signature-dependent	Yes (hash-based)
MEV window	Yes	No	No	No	No

8.3 Polkadot BABE — Colton’s pointer

Colton suggested investigating whether **Polkadot’s consensus-derived randomness** could substitute for or supplement drand/NIST. Specifically: Polkadot uses BABE (a VRF-based block production protocol), which produces a stream of publicly-readable randomness as a byproduct.

Key insight: Polkadot’s setup is multi-participant VRF — exactly what we rejected when considering building one ourselves. The reason it works for Polkadot: they already run a validator-network for *consensus* (their core job), so the randomness oracle is essentially free for them as a byproduct. We can’t justify building such a network for randomness alone, but we *can* consume Polkadot’s existing one.

Bittensor is built on Substrate (the Polkadot SDK), so if we’re running as a Bittensor subnet, BABE-derived randomness might be available in-chain natively — reducing external HTTP dependencies on drand/NIST. Worth investigating in the **quip-protocol-rs** repo Colton mentioned.

Important nuance: when Colton says “Polkadot’s randomness,” he means *consuming* their oracle, not *building* one ourselves. The validator-network is Polkadot’s problem to operate, not ours.

8.4 The fundamental “why no secret key” argument

The chain is tight:

1. A VRF requires a secret key.
2. The key has to be held by someone.
3. Whoever holds it controls every output.
4. The protocol’s security reduces to trusting that party.

The way out — many keyholders in parallel — only moves the trust problem to a validator network with its own infrastructure burden.

Our design avoids the chain entirely by having no secret key whose secrecy the seed depends on. Signing keys exist for *attribution* (you can’t repudiate your commit; drand’s signatures verify pulse authenticity), but the seed’s *unpredictability* depends only on the hash function and on at least one source being honest. **Authentication keys and unpredictability are decoupled** — that’s the architectural shift.

9. Mehdi’s deck — what he added beyond our earlier conversation

Mehdi’s June 2026 deck (titled “QVRF Project — Seed Generation Design”) substantively aligned with our prior conversation but added two pieces:

9.1 The PoW-style public puzzle

After commits close, the puzzle is defined:

```
B = H(drand_pulse || nist_pulse || prev_finalized_round_hash)
puzzle = H("quip-puzzle-v1" || round_id || parameter_commitment
          || commit_root || B)
```

Anyone in the world can try nonces until $H(\text{puzzle} || \text{nonce}) < \text{target}_T$. First valid solution before cutoff included; otherwise fallback $H(\text{"NO_SOLUTION"} || \text{puzzle})$.

The puzzle does three jobs at once:

- **Permissionless liveness anchor:** anyone in the world can keep the protocol producing output without prior registration or commit-phase participation. The other channels (operator, pool, beacons) all require pre-arrangement.
- **Additional bias resistance:** compute cost to attempts at seed manipulation.
- **Widens participation:** contributors who weren’t at commit time can still contribute.

“Permissionless liveness anchor” = a contribution channel anyone can use without prior arrangement. The protocol’s liveness no longer depends on any specific party showing up — only on the weak assumption that *someone, somewhere* will do the work.

9.2 The chained transcript

The `prev_finalized_round_hash` field in slide 18 links each round’s seed to the previous round’s finalized state. This is hash-chain integrity *baked into the seed*, not just in the audit log. An attacker can’t manipulate the current round without invalidating all subsequent rounds.

9.3 Other Mehdi deck items worth raising

- **Hash-graph-based timing for Leg 5** (replacing wall-clock) — slide 6 annotation. A potentially clever timing-enforcement mechanism, but raises its own questions about propagation latency and adversarial manipulation.
- **PoW difficulty mechanism for Leg 2 parameters** — slide 6 annotation. The PoW machinery may be doing *triple duty* across Legs 1, 2, and 5. If so, this is a substantial architectural move worth understanding fully.
- **“Classical VRF Participants” label** still on slide 6 — likely vestigial from earlier diagram versions. Worth asking what those participants are in the new design.

9.4 Open questions for the design meeting with Mehdi

- Is the puzzle **required every round** or **fallback-only** when other channels fail? (Materially changes protocol character.)
- What’s the puzzle’s difficulty target? Who pays the compute cost? (Bitcoin-like incentives required, or unpaid solvers?)
- How does hash-graph-based timing (Leg 5) actually work? What’s the open H/SHAKE-256 question?
- How does PoW difficulty drive Leg 2 parameter selection?

- Are these all “decisions” or still “proposals for discussion”?
 - Pin the exact hash primitive: SHA-3-256 vs SHAKE-256.
 - Confirm participant terminology — what are “participants” in the new design?
-

10. Key corrections made during the session

Three substantive corrections made — worth preserving so they don’t get re-introduced into future communications.

10.1 Two-sided F_{XEB} check is OUR addition, not Liu’s

What I’d been saying: Liu uses a two-sided check.

What’s actually true: Liu uses a **one-sided** check ($F_{\text{XEB}} \geq \chi$). Their compute-rational adversary wouldn’t waste resources scoring $\gg 1$. Our v0.1+ two-sided check is a **deliberate defence-in-depth addition** by us — catches threats outside Liu’s adversary model.

Status: corrected in `certified_randomness_story.pdf`.

10.2 AH isn’t “broken by amortization”

What I’d been saying: Aaronson-Hung’s structure (one circuit, many shots) is “broken by amortization”; Liu’s structure “fixes” it.

What’s actually true: AH’s structure has **correlated shots** (the shots share a circuit). AH’s proof handles this correctly via the **Entropy Accumulation Theorem (EAT)** — the bound is *correct*, not broken. Liu’s structure (many distinct circuits, one shot each) has **independent rounds**, which enables cleaner i.i.d. concentration bounds and a **tighter** result for the same parameters.

The switch in v0.4 wasn’t a bug fix — it was a structural choice yielding a tighter bound and a simpler proof.

Status: corrected in `leg5_verification_design.pdf` slides 8, 9, 10 and `leg5_speaker_notes.md` slides 8, 9, 10.

10.3 `compute_q_min` isn’t a placeholder

What gets undersold: “placeholder” implies no work done.

What’s actually true: `compute_q_min` is a *real implementation* of Liu’s adversary-model machinery — it implements the hypergeometric-tail optimisation, takes the right inputs, returns plausible integers. It’s **uncalibrated**, not absent. Three numerical conventions need reconciliation against Liu’s Zenodo code. The skeleton is right; the constants need tuning.

Status: correctly framed in `leg5_verification_design.pdf` slide 25 and speaker notes.

10.4 QVRF → QSeed (with caveat)

Mehdi’s deck still calls the project “QVRF,” which is now misleading since there’s no VRF in the protocol. Our deck was renamed to **QSeed** as a working label — with the honest caveat that QSeed is *narrower* than what the deck actually covers (the deck is about verification + certification, not seed generation). Cleanest end-state probably ties the umbrella name to the *output* (Certified Randomness / QPoW) rather than to a specific leg. Worth aligning with Mehdi.

11. Documents produced

All under /mnt/user-data/outputs/:

File	Purpose	Length
certified_randomness_story.pdf	Story-paced math walkthrough — full certification chain	~11 pages
certified_randomness_story.tex	LaTeX source for the above	—
leg1_redesign_story.pdf	Story-paced explainer for the new commit-reveal + beacons construction	~15 pages
leg1_redesign_story.tex	LaTeX source	—
leg5_verification_design.pdf	Companion design deck to Mehdi’s Leg 1 deck — verification & certification	28 slides
leg5_verification_design.tex	Beamer source	—
leg5_speaker_notes.md	Presenter notes for the Leg 5 deck	~28 sections
porter_thomas_report.pdf	Rigorous derivation of Porter-Thomas, F_XEB moments, certified-entropy math	15 pages
porter_thomas_report.tex	LaTeX source	—
ah_vs_liu_reference.pdf	Side-by-side variance analysis: AH vs Liu sampling regimes, with the central law-of-total-variance derivation	10 pages
ah_vs_liu_reference.tex	LaTeX source	—
quip_design_notes.md / .pdf	This document — consolidated design notes	18 pages
meeting_prep_mehdi.md	Notion-style meeting prep doc — questions and suggestions for the Leg 1 design meeting	~2700 words
leg5_qmin_liu.py	The calibrated Q_min implementation, reproducing Liu’s notebook exactly	—
verify_against_liu_data.py	End-to-end verification script against Liu’s experimental data	—
aggregated_probs.npy	Liu’s published 1,522 precomputed validation probabilities (from their Zenodo archive)	12 KB
verification_package.tar.gz	Bundled verification package (script + data + README) for drop-in use in the audit-trail repo	—

File	Purpose	Length
quip-protocol-history.tar.gz	Internal-only audit-trail repository, v0 → v0.4 lineage with full documentation	3 MB, 51 files

The two story-paced PDFs together cover the entire protocol’s design rationale: `leg1_redesign_story` for the seed layer, `certified_randomness_story` for everything downstream. The Leg 5 deck pairs with Mehdi’s QVRF/QSeed deck for the design conversation. The Porter-Thomas report and AH-vs-Liu reference are the mathematical reference docs; this document is the consolidated overview.

12. Outstanding items and the critical path

(Section revised end-of-session: $Q_{\min}$ calibration removed as an open item — it’s closed and end-to-end verified. Other items re-prioritised based on what remains.)

12.1 Closed in this session

- **DONE: $Q_{\min}$ calibration** — reconciled against Liu’s reference notebook (Table2-bounds-on-extractable). Calibrated module is `leg5_qmin_liu.py`. Reproduces $Q_{\min} = 1297$ exactly.
- **DONE: End-to-end verification against Liu’s experimental data** — `verify_against_liu_data.py` loads Liu’s `aggregated_probs.npy` and reproduces $F_{\text{XEB}} = 0.32$, $Q_{\min} = 1297$, certified entropy = 71,313 bits, extractable bits = 71,273 bits. All match to the digit.
- **DONE: AH-vs-Liu structural justification** — formal variance analysis in `ah_vs_liu_reference.pdf` documents why v0.4’s structure was the right choice.

12.2 Open math / code items (bounded engineering)

- **Timing enforcement (Leg 5)** — design decision among hash-graph propagation / drand pulse ordering / cryptographic timestamping. No math problem; gated on the Mehdi meeting.
- **NIST STS uniformity sanity-test** on Toeplitz extractor output — small bounded follow-up, useful for external audit.
- **Hash primitive choice** — SHA-3-256 vs SHAKE-256 for canonical rule. Open per Mehdi’s deck.
- **Two-sided F_{XEB} upper-bound tightening** (optional) — our two-sided check inherits Liu’s one-sided bound. A direct proof for the two-sided variant would be slightly tighter; not blocking.

12.3 Open modelling decisions

- **Choose our own adversary model** — see Section 4.8. Three reasonable options (inherit Liu’s verbatim, tighten, loosen); decision belongs in conversation with Colton and Mehdi. Not a math problem; a product/security tradeoff.

12.4 Leg 1 redesign

- Send Mehdi the response (drafted, ready when convenient).
- Schedule design meeting on open questions (puzzle scope, hash-graph timing, Leg 2 PoW difficulty, participant terminology, byte-level canonical rule, fallback rules, commit-and-reveal ordering).

- Implement the new Leg 1 once design is closed out.

12.5 Coordination items

- **Reply to Colton** about the Polkadot/BABE pointer and the quip-protocol-rs repo investigation (acknowledge and commit to look at; raise with Mehdi at next meeting).
- **Loop in author** of the parallel ETH+RCS-VRF architecture sketch with Mehdi's re-design so the architecture is reconciled.
- **Umbrella project naming** — align with Mehdi on QVRF/QSeed/QPoUW/QCR. Defer to follow-up.

12.6 Critical-path items (Colton-owned, non-math)

- **Real QPU access for Leg 4** — vendor conversations (IonQ / Quantinuum).
- **Customer willingness-to-pay validation** before committing to QPU spend.

These remain the live items gating real product. With the math layer now closed and end-to-end verified, and the seed-generation layer in active redesign, the strategic blockers are entirely on the hardware-and-customer side.

12.7 Investigation items (per Colton's reply)

- **quip-protocol-rs repo** — check what's already there on randomness.
- **Polkadot/BABE randomness consumption** — could substitute for or supplement drand/NIST. Failure-mode independence must be preserved.

13. Reading resources

13.1 For understanding VRFs (gentle to deep)

1. **Chainlink's "Verifiable Random Function (VRF) — Explained"** — chain.link/education-hub/verifiable-random-function. Developer-friendly intro.
2. **Wikipedia article on Verifiable Random Function** — middle-ground reference.
3. **Algorand's blog posts** — "Algorand Releases First Open-Source Code of Verifiable Random Function" + "Unpredictable Randomness Thanks to Verifiable Random Functions" on the developer portal. Grounded in a real use case (consensus sortition).
4. **RFC 9381 — "Verifiable Random Functions (VRFs)"** — the standardisation document. Defines RSA-FDH-VRF and EC-VRF.
5. **Dodis & Yampolskiy, "A Verifiable Random Function With Short Proofs and Keys"** (PKC 2005) — cs.nyu.edu/~dodis/ps/short-vrf.pdf. Clean academic treatment.
6. **Micali, Rabin, Vadhan (1999), Verifiable Random Functions** (FOCS '99) — the original paper. Mostly historical interest now.

Suggested reading order:

- Basic working understanding: Chainlink + Wikipedia + Algorand blog (~30–45 minutes).
- Enough to debate Mehdi confidently: add the Algorand developer portal piece + skim RFC 9381 sections 1–3 (+1 hour).
- Deep theoretical grounding: add Dodis-Yampolskiy and MRV99 (half a day).

Practical tip: work through Schnorr signatures before tackling EC-VRF math. EC-VRF is essentially Schnorr repurposed.

13.2 PQ-VRFs (relevant for “why we didn’t just swap PQ-VRF in”)

- **LB-VRF** — “**Practical Post-Quantum Few-Time Verifiable Random Functions**” (lattice-based, Algorand’s direction).
- **X-VRF** — “**Post-Quantum Verifiable Random Function from Symmetric Primitives**” — note its uniqueness proof was broken at FC 2024-25. Cautionary tale.

13.3 Liu et al. and adversary model

- **Liu et al. (2025)**, *Certified randomness using a trapped-ion quantum processor* — the paper we’re implementing.
- **Liu Zenodo archive** (DOI 10.5281/zenodo.12952178) — the reference code base. 3 GB total; the security-analysis codebase is small (under 10 MB). Required for Q_min calibration.
- **Aaronson & Hung (2023)**, *Certified Randomness from Quantum Supremacy* (STOC) — the predecessor protocol whose AH structure we initially implemented and switched away from in v0.4.

13.4 NIST presentation

- **Niroula’s NIST presentation** (April 2025) — `csrc.nist.gov/.../crypto-club-20250430--pradeep--certi`. Alternative path for understanding Liu’s framing.

Appendix — Working terminology

- **Q_min**: minimum number of honest quantum rounds the adversary cannot avoid. Floor on certifiable quantum content.
- **F_XEB**: linear cross-entropy benchmark. Verification score; one number per audit batch.
- χ (chi): F_XEB acceptance threshold. Liu uses 0.3.
- Φ_A : adversary’s classical faking budget within the deadline.
- ϵ_s (epsilon_s): smoothing parameter for certified min-entropy. Liu uses 2.5×10^{-7} .
- ϵ_{sou} : total smoothing for extracted bits (extractor + entropy). Liu uses 10^{-6} .
- **t_threshold**: per-circuit response deadline. Liu uses ~2 seconds.
- **n**: qubits per circuit. Liu uses 56.
- **N**: 2^n , number of possible bitstrings per circuit.
- **M**: number of distinct circuits per round. Liu uses 30,010.
- **m**: audit-subset size, the circuits the verifier classically simulates to compute F_XEB.
- **H_2**: collision (2-Rényi) entropy. For our case, equal to $n - 1$.
- **Haar**: the unique uniform measure over unitaries.
- **t-design**: ensemble reproducing first t moments of Haar. We need 2-design.
- **RCS**: random circuit sampling. The recipe for approximating a 2-design with shallow circuits.
- **Porter-Thomas**: the output probability distribution of Haar-random circuits.
- **MEV**: maximum extractable value. Class of attacks where one party’s positional advantage lets them extract value.
- **EAT**: Entropy Accumulation Theorem. Cryptographic machinery for bounding entropy over correlated rounds.
- **LHL**: Leftover Hash Lemma. Cryptographic result bounding distance from uniform after randomness extraction.
- **drand**: federated public randomness beacon (Cloudflare, EPFL, Protocol Labs, et al.).
- **NIST randomness beacon**: US government public randomness beacon.
- **SHAKE-256**: extendable-output hash function from the SHA-3 family.

- **EC-VRF**: elliptic-curve verifiable random function (RFC 9381).
- **BABE**: Polkadot's block production protocol, VRF-based.